

Scenario	Prompt
Task Assessment	# CANDIDATE TASK LIST # {{ candidate tasks }} # GOAL # Based on the provided # CANDIDATE TASK LIST # and the user’s request described in the # STEP #, generate a score dictionary to assess each task’s problem-solving abilities. The output must be in a strict JSON format, like: { “candidate task name 1”: score, ... }. # REQUIREMENTS # 1. The keys of the generated score dictionary must align with the provided candidate tasks, and you should output scores for all candidate tasks. 2. The “score” field denotes a concrete score that assesses whether each task can solve the given step’s demand. The score should be in the range of [1, 2, 3, 4, 5], where a higher score indicates better task-solving and matching abilities. 3. Carefully consider the user’s intention in # STEP # to assign the score. If the # STEP # contains a candidate task, its score should be >= 3. # EXAMPLE # {{ in-context learning examples }} # STEP # {{ step description }} Now please generate your result in a strict JSON format: # RESULT #
Path Selection	# GOAL # Based on the provided # USER REQUEST # and initially inferred # STEPS #, select the best path solution list from # SOLUTION LIST #. The selected solution should be the one that can perfectly solve the user’s request and strictly align with the inferred steps. The output must be in strict JSON format, like: { “best_solution”: [list of invoked tasks] } # REQUIREMENTS # 1. Carefully analyze both the user’s request and previously inferred task steps. Select the best solution that can perfectly follow the inferred steps and solve user’s request. Do not change their corresponding sequences. 2. Make sure that each task in the final solution list exists in the valid # TASK LIST # {{ task list }}. # USER REQUEST # {{ user request }} # STEPS # {{ steps }} # SOLUTION LIST # {{ list of searched solutions }} Now please generate your result in a strict JSON format: # RESULT #